Tradução PT-BR — Capítulo 3

Arquivo original: TG2_Bruno_Silveira.pdf

Capítulo traduzido: 3 Related Work (Trabalhos Relacionados), Seções 3.1 e 3.2.

3 TRABALHOS RELACIONADOS

Este capítulo tem como objetivo explorar o panorama atual de pesquisas relacionadas ao benchmarking de latência de ANNs em dispositivos pequenos. A Seção 3.1 discute diferentes abordagens de Busca de Arquiteturas Neurais (NAS) visando otimizar a latência do modelo, com exemplos de trabalhos que realizam medições reais e de trabalhos que utilizam proxies. A Seção 3.2 compara ferramentas desenvolvidas para automatizar a implantação e o benchmarking de modelos especificamente em microcontroladores.

3.1 NAS em TinyML

Na literatura recente, alguns trabalhos exploram como HIL (Hardware In the Loop) — isto é, o uso de medições em hardware real em vez de simulações completas — pode ser utilizado para desenvolver DNNs com melhor desempenho. Os autores em (ESSA et al., 2023) propõem uma abordagem com DNN para controlar a velocidade de um Veículo Elétrico Híbrido (HEV) em substituição ao algoritmo padrão de Controle Preditivo por Modelo, que é computacionalmente exigente. Eles propõem três arquiteturas de redes neurais criadas manualmente e comparam seu desempenho tanto em ambiente simulado quanto no microcontrolador real. Em métricas-chave, especificamente ITAE (Integral Time Absolute Error) e IAE (Integral Absolute Error), observa-se que as simulações indicariam uma arquitetura com desempenho superior, mas o HIL mostra resultados diferentes, em que uma arquitetura ligeiramente inferior na simulação passa a superar as demais quando medida no dispositivo. Por fim, os experimentos HIL ajudaram a determinar qual das três arquiteturas propostas era melhor, com desempenho considerado recomendado para uso em controle de velocidade de HEVs.

Além de testar modelos criados manualmente em hardware real, alguns trabalhos experimentaram a ideia de integrar HIL e Busca de Arquiteturas Neurais. Os autores de (TAN et al., 2019) propuseram usar medições reais de latência de inferência em smartphones Google Pixel 1 como parâmetro a ser otimizado em uma busca multiobjetivo, justificando que proxies de aproximação, como número de operações em ponto flutuante, não produzem resultados suficientemente precisos. Nesse projeto, chamado MnasNet, eles produziram cerca de 8.000 arquiteturas candidatas e realizaram um mini-treinamento inicial de apenas 5 épocas no conjunto ImageNet, com o objetivo de economizar tempo em vez de treinar completamente cada candidato. Todos os modelos parcialmente treinados foram então perfilados no dispositivo: compilados usando TensorFlow Lite, enviados ao aparelho e medidos quanto ao tempo de inferência. Essa busca, mini-treinamento e profiling levou 4,5 dias; e, embora os autores afirmem que 64 dispositivos TPUv2 foram usados para busca e treinamento, não é explicitado quantos smartphones foram usados, tornando difícil inferir quanto de sobrecarga cada medição on-device adicionou ao processo. Ao final da busca, os 15 modelos de melhor desempenho foram treinados completamente; o mais rápido deles apresentou, com acurácia semelhante, um speed-up de cerca de 2× quando comparado a outros modelos estado da arte rodando no mesmo hardware.

A partir das ideias do MnasNet, os autores em (CAI; ZHU; HAN, 2019) propõem duas formas de acelerar o processo de NAS para CNNs, que, combinadas, eles chamaram de ProxylessNAS. Eles criticam o uso de proxies de tarefa em NAS, afirmando que resultados subótimos podem ocorrer quando se utilizam atalhos para economizar tempo, como treinar arquiteturas candidatas por poucas épocas ou treinar inicialmente em tarefas similares, mas mais simples (por exemplo, CIFAR-10 em vez de ImageNet). Para evitar tais proxies de tarefa e, ao mesmo tempo, obter ordens de grandeza de aceleração na busca, eles propõem a ideia de treinar uma rede gigante superparametrizada que contém todos os blocos possíveis (operadores) a serem usados. Quando uma arquitetura candidata é produzida, ela reutiliza os pesos desses mesmos blocos

da rede superparametrizada, economizando tempo ao pular o treinamento do zero de milhares de modelos candidatos, enquanto utiliza métricas diretamente ligadas à tarefa-alvo. Com isso, eles obtêm um custo de busca de apenas 200 horas de GPU, muito menor do que as 40.000 horas de GPU alegadas para o MnasNet, mantendo latência e acurácia semelhantes.

O ProxylessNAS também afirma que medir latência on-device como no MnasNet demanda tempo e infraestrutura, pois seriam necessários muitos dispositivos móveis. Em vez disso, eles treinam um modelo que estima a latência no dispositivo móvel-alvo, precisando de apenas um dispositivo para isso, e usam esse estimador como métrica de latência. A Figura 3.1 mostra tempos de latência medidos no Google Pixel 1 versus a latência estimada pelo modelo estimador, indicando que a aproximação é bastante próxima da realidade. Um resultado interessante desse artigo é que, ao otimizar a busca considerando latência para diferentes hardwares-alvo (GPU, CPU e mobile), os melhores modelos para cada hardware apresentam arquiteturas distintamente diferentes, destacando quão únicos são os dispositivos ao executar inferência, como mostrado na Figura 3.2.

Um aspecto importante é que, tanto no MnasNet quanto no ProxylessNAS, algum tipo de workflow de implantação de modelo foi usado para testar o modelo no dispositivo-alvo. No caso do MnasNet, infelizmente o processo não é detalhado; mas, considerando que 8.000 modelos candidatos foram implantados em telefones, é razoável assumir que houve esforço significativo de engenharia de software para automatizar o processo. Já no ProxylessNAS, embora tenha sido usado um estimador de latência em vez de implantar todos os candidatos, ainda assim foi necessário realizar um número de implantações e medições de latência para treinar o estimador. A falta de detalhes desse processo prejudica a reprodutibilidade do experimento. Além disso, ambos os trabalhos automatizam implantação para CPUs de smartphones, que executam sobre um sistema operacional, e não em bare-metal como MCUs de baixo consumo, que são o foco desta pesquisa. Em suma, esses dois artigos mostram um foco reduzido na metodologia prática, significando que outros trabalhos que busquem reproduzir seus experimentos precisariam redesenvolver esses frameworks de implantação do zero.

Como exemplo de NAS com latência como métrica e que tem como alvo dispositivos TinyML, podemos citar (LIBERIS; DUDZIAK; LANE, 2021). Esse trabalho, chamado µNAS, propõe um framework de NAS projetado para buscar adequadamente no espaço de arquiteturas de TinyML, com maior granularidade, consideração de restrições apropriadas a microcontroladores e incluindo pruning para produzir modelos ainda menores. O µNAS considera apenas modelos completamente quantizados em int8 e tenta reduzir o pico de uso de RAM dos modelos, bem como a latência de inferência. Entretanto, eles não realizam medições reais de latência no MCU, optando por usar o número de MACs como métrica proxy. Para justificar essa escolha, eles plotam o número de MACs versus o tempo de inferência medido em uma NUCLEO-H743ZI2 de 1.000 modelos aleatórios do espaço de busca, exibindo um índice R^2 de 0,975. O gráfico mostra uma correlação clara, embora a seção menos linear entre 0 e 20M MACs não seja mencionada, o que poderia prejudicar a qualidade da busca nessa região. O µNAS encontrou modelos com até 13× menos pico de uso de RAM e com o mesmo número de MACs e acurácia que outros modelos estado da arte, além de modelos com ao menos 2× menos MACs e acurácia comparável.

A Tabela 3.1 resume os trabalhos apresentados e os compara à ferramenta proposta neste trabalho.

Tabela 3.1: Resumo de trabalhos relacionados de NAS para TinyML

Artigo | Plataforma-alvo | Estimativa de latência | Workflow automatizado | Dados de tempo (ESSA et al., 2023) | MCU | Medições reais | Não | Não MnasNet | CPU mobile | Medições reais | Sim | Não ProxylessNAS | CPU mobile | Modelo preditor | Sim | Não µNAS | MCU | Número de MACs | Sim | Não

B.A.L.A.S | MCU | Medições reais | Sim | Sim

Fonte: autor

3.2 Ferramentas para profiling de modelos em MCUs

Em relação a ferramentas desenvolvidas para implantar e medir automaticamente modelos de aprendizado de máquina em microcontroladores, podemos citar alguns trabalhos existentes. O MLonMCU (KEMPEN et al., 2023) é uma ferramenta de benchmarking para MCU baseada em Python, que disponibiliza um pacote Python e uma ferramenta de linha de comando (CLI) para usuários. Ela suporta dois frameworks de TinyML, cada um com algumas variações: TFLM tanto com o interpretador padrão quanto com o pré-processador TFLMC; e TVM com o executor de grafo padrão, o executor Ahead-Of-Time (AOT) ou a extensão AOT+, que realiza otimizações de planejamento de memória. Entre os pontos fortes desse trabalho estão o fato de ter sido desenvolvido de forma orientada a objetos com interfaces extensíveis, permitindo extensões por outros desenvolvedores, e também o fato de ter sido validado com sucesso em todos os quatro modelos padrão do MLPerfTiny (BANBURY et al., 2021). Além disso, o MLonMCU é singular por compartilhar dados de tempo, com tempo médio de build de 17 segundos em 20 execuções de modelos. Deve-se mencionar, contudo, que o fator tempo total é apenas brevemente discutido e sobre apenas 20 modelos, e que o MLonMCU armazena as entradas dos modelos em ROM em vez de enviá-las em tempo de execução, o que consome ROM que poderia ser usada pelos pesos do modelo. Há também falta de sistemas totalmente integrados de build e deploy (isto é, o usuário precisa implementar o próprio sistema; não há automações pré-desenvolvidas).

No mesmo sentido, podemos analisar o MLino Bench (BACIU; STIENS; SILVA, 2024), que se propõe a ser uma ferramenta de benchmarking mais completa para TinyML. O MLino Bench oferece um pipeline totalmente automatizado baseado em Python com a opção de treinar um modelo como etapa inicial ou carregar um modelo pré-treinado, além da opção de quantizá-lo. Essa ferramenta suporta redes neurais com TFLM e também modelos clássicos de ML como Random Forests e Support Vector Machines usando uma mistura de frameworks: emlearn, skearn-porter, embml, micromlgen e m2cgen. Após isso, o modelo é implantado automaticamente no dispositivo-alvo, onde é avaliado quanto à acurácia no alvo, tempo de inferência, uso de ROM e uso de RAM. A implementação de um processo totalmente automatizado, do treinamento ao deploy, é um ponto forte; contudo, pode-se notar que o suporte para a parte crucial final do pipeline, o deploy, está disponível apenas para placas Arduino, Atmel SAM, chipKIT PIC32, Teensy e Robotis OpenCR.

Outra ferramenta que oferece benchmarking de modelos TinyML em MCUs é o U-TOE (HUANG et al., 2023). Essa ferramenta mede RAM, ROM e latência de modelos completos, bem como de operadores individuais usando o framework uTVM. Sua grande vantagem sobre ferramentas similares é a possibilidade de profiling remoto via TCP (Transmission Control Protocol), embora uma desvantagem seja que as entradas de teste são sempre aleatórias, e não entradas reais enviadas para a placa, o que impossibilita avaliações de acurácia. Um ponto forte é a disponibilidade de workflows totalmente automatizados para os dispositivos testados, bem como resultados detalhados de benchmarking das medições, embora faltem dados de tempo para build e deploy.

A Tabela 3.2 resume as ferramentas apresentadas, evidenciando as diferenças entre elas e o B.A.L.A.S.

Tabela 3.2: Resumo de ferramentas relacionadas de benchmarking de latência em TinyML

Ferramenta | Frameworks | Workflow automatizado | Dados de tempo | Método de entrada MLonMCU |

TFLM, µTVM | Não | Pouco | ROM MLinoBench | TFLM, scikit-learn | Sim | Não | Serial U-TOE | µTVM | Sim | Não | Aleatórias B.A.L.A.S | TFLM | Sim | Detalhado | Serial

Fonte: autor

Em síntese, após a revisão de trabalhos relacionados, fica claro que medir em hardware produz os resultados mais precisos para escolher entre arquiteturas candidatas de redes neurais, mesmo ao projetar modelos manualmente. Integrar implantações e medições reais ao loop de NAS é uma linha de pesquisa atual, embora poucos artigos explorem isso diretamente, e estimadores de latência ainda representem grande parte dessas pesquisas. Seja para projetar manualmente arquiteturas, para integrar com NAS, ou para treinar um estimador de latência, a necessidade de um processo totalmente automatizado de implantação e benchmarking em MCUs é evidente. Entre as ferramentas revisadas, há uma lacuna de uma ferramenta totalmente automatizada que suporte um microcontrolador ARM Cortex-M moderno, bem como uma lacuna de documentação sobre as etapas de implementação de tal ferramenta e, principalmente, sobre quanto tempo leva para realizar medições reais em modelos.